Lecture Indexer: Prototype Review

CS 6030 Cloud Computing — Semester Project, Milestone 2

Theodore Podewil
Dept. of Computer Science
Western Michigan University
Kalamazoo, MI, USA

Joshua Jubenville
Dept. of Computer Science
Western Michigan University
Kalamazoo, MI, USA

Andrew Wojciechowski
Dept. of Computer Science
Western Michigan University
Kalamazoo, MI, USA

TABLE I
MILESTONE 2 DELIVERABLES AND EVIDENCE

Deliverable	Evidence
Initial deployment	Terraform `envs/prod` applied on 2026-10-18 (52 resources, GCP and Azure); live frontend and API (Sec. III)
CI/CD pipeline	6-job CI on every push; one-approval release with automatic rollback; releases `prod-2026-10-18.1`, `-10-19.1`, `-10-27.1` (Sec. IV)
Basic functionality	End-to-end test passed in production; 9 manual test cases; 126 automated tests passing (Sec. V)

***Abstract*—Milestone 2 asks for an initial deployment, a CI/CD pipeline, and basic functionality. Lecture Indexer (LI) meets all three. Terraform provisions the production environment on Google Cloud and Microsoft Azure. A GitHub Actions pipeline tests every push and releases every merge to `main` after one human approval, with automatic rollback. It has shipped three tagged releases. In production, a professor signs in with Google and registers a YouTube lecture, LI indexes its auto-generated captions, and a student searches the course and jumps to the moment a word is spoken. This report maps each deliverable to the evidence in our repository and explains where the build differs from the Milestone 1 proposal, and why.**

***Index Terms*—cloud-native, multi-cloud, Kubernetes, CI/CD, Terraform, GitHub Actions.**

I. Introduction

Our Milestone 1 proposal described LI, a service that retrieves the timed captions of recorded lectures from the YouTube Data API, indexes them with PostgreSQL full-text search, and returns deep links such as `?t=202` to the moment a term is spoken [1]. Table I summarizes how we met each Milestone 2 deliverable. Sections III–V give the evidence for each one. All source code, workflow runs, and releases are public at https://github.com/SisyphusRex/cloud-computing-semester-project.

II. As-Built Architecture

Fig. 1 shows the system we deployed. The React SPA is served by nginx on Cloud Run. It calls two Flask services, the Ingest API and the Search API, on a GKE Autopilot cluster. A global HTTPS load balancer with a Google-managed certificate routes `/ingest/*` and `/search/*` to them. Each service has its own Horizontal Pod Autoscaler (1–4 replicas at 70% CPU) and a Cloud SQL Auth Proxy sidecar that connects to a private-IP Cloud SQL for PostgreSQL instance using IAM database authentication. Images are stored in Artifact Registry, and the database admin password in Secret Manager. Pods reach Google APIs through Workload Identity, so no service-account keys exist.

A. Changes from the proposal

- **Frontend on Cloud Run, not Azure Static Web Apps.** Our Azure for Students subscription allows only five regions, and Static Web Apps is offered in none of them. The frontend therefore runs as an nginx container on Cloud Run, which scales to zero [2].
- **Azure region.** Azure refused `eastus2` (`RequestDisallowedByAzure`), so all Azure resources are in `northcentralus`.
- **Invite-only sign-in.** We started with a `PROFESSOR_EMAILS` allowlist. It published email addresses in a public repository and required a release for every role change. Roles now live in the database: admins invite users and set their roles on an Admin page [3].

The multi-cloud requirement still holds. Azure hosts the CI deploy identities, which are federated with GitHub over OIDC, and the production resource group for the off-site backups (Section VII).

III. Deliverable 1: Initial Deployment

A. Infrastructure as code

All infrastructure is Terraform, in two stages [4]:

1) **`bootstrap/`** creates the CI identities: a GCP Workload Identity pool and the service accounts `github-terraform`, `github-deploy`, and `github-plan`, plus the Azure managed identities `id-li-github` and `id-li-github-plan`. Each has federated credentials scoped to our repository and its `prod` environment. State lives in a versioned GCS bucket with public access blocked.
2) **`envs/prod`** composes reusable modules (`gcp-network`, `gke`, `cloudsql`,

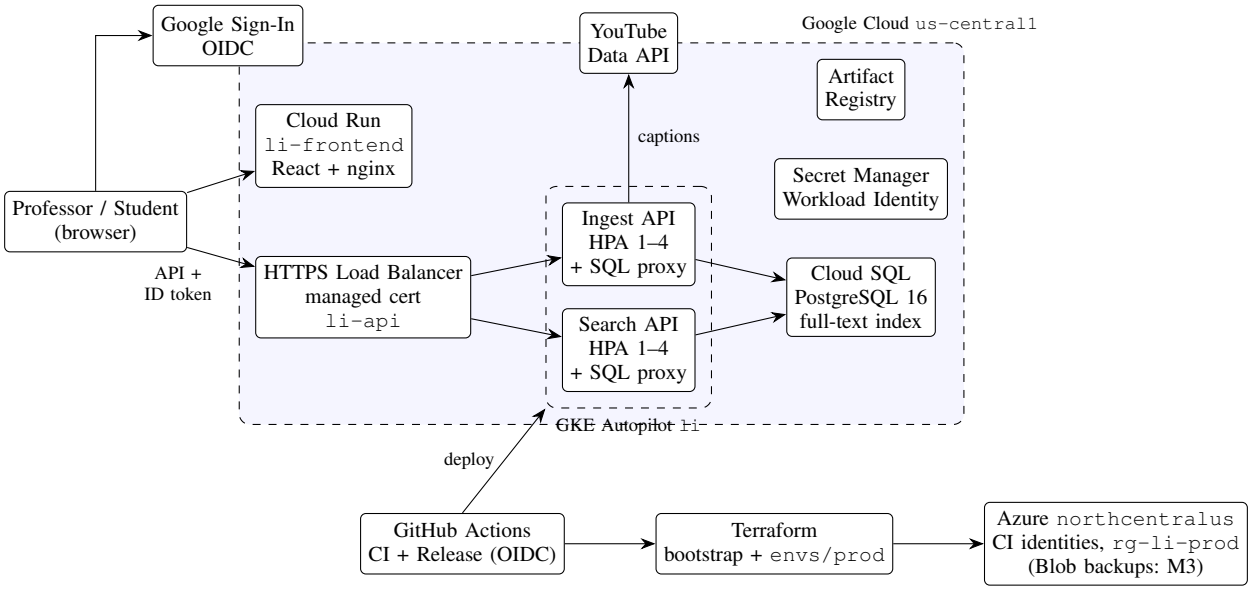


Fig. 1. As-built architecture of the production environment. Dashed boxes mark cloud and cluster boundaries. Azure holds the CI identities and the resource group for the nightly backups, which arrive in Milestone 3.

TABLE II
PRODUCTION RESOURCES BY PROVIDER

Provider	Resources
Google Cloud	VPC `li` + Cloud NAT; GKE Autopilot `li`; Cloud SQL `li-pg`; Artifact Registry `li`; Secret Manager; Cloud Run `li-frontend`; global IP `li-api`; workload service accounts; Workload Identity pool
Microsoft Azure	`rg-li-ci` with the two federated CI identities; `rg-li-prod` for backups
GitHub	`prod` and `prod-plan` environments, 11 repository variables, required reviewers

`artifact-registry`, `secrets`, `cloud-run-frontend`) into 52 resources. We applied it on 2026-10-18; GKE took 7 min 38 s and Cloud SQL 13 min 4 s. A fresh plan shows no changes.

CI runs `terraform validate` and a mocked `terraform test` suite (3 tests) on every push.

Kubernetes manifests are Kustomize bases with a `prod` overlay (`deploy/k8s/`). They define Deployments, Services, HPAs, a default-deny NetworkPolicy, the Ingress with a ManagedCertificate, and a migration Job that runs numbered SQL migrations before each rollout. `make k8s-check` validates the rendered overlays against Kubernetes schemas.

B. Deployed environment

Table II lists what runs where. The live endpoints are:

- Frontend: https://li-frontend-byshtpdfha-uc.a.run.app
- API: https://34-128-137-19.sslip.io. Without a sign-in, `/search/courses` answers `401`.

Figs. 2–4 show the running workloads and the Azure identities.

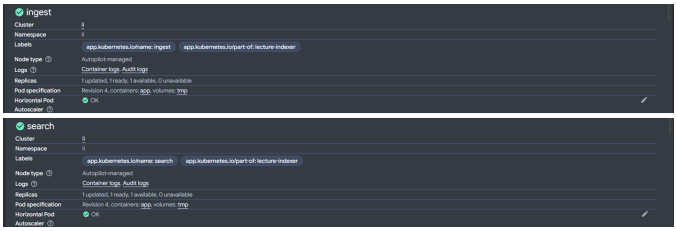


Fig. 2. The `ingest` and `search` Deployments on GKE Autopilot cluster `li`, namespace `li`: replicas ready and Horizontal Pod Autoscaler OK.

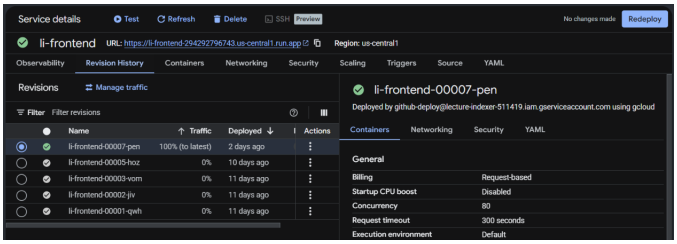


Fig. 3. Cloud Run service `li-frontend`: one revision per release, deployed by the `github-deploy` service account, with all traffic on the latest.

Cost control. `make cloud-pause` scales the API pods to zero, deletes the load balancer, and stops Cloud SQL. It keeps all data and the API hostname, and `make cloud-resume` reverses it. Idle cost drops from about $40–60 to about $9 per month. Terraform ignores Cloud SQL's activation state, so an apply never restarts a paused database.

C. Problems found and fixed

Bringing the environment up exposed three problems. We fixed each one in code, so that a fresh deployment works the first time:

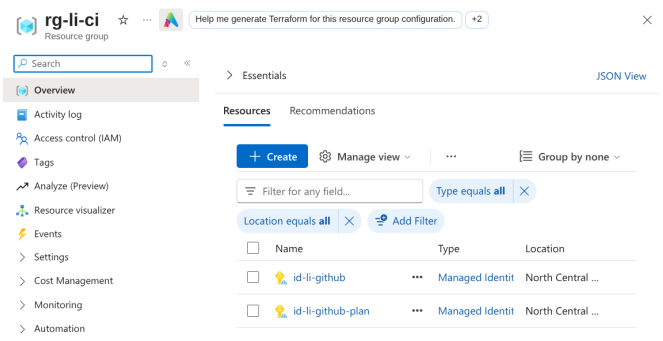


Fig. 4. Azure resource group `rg-li-ci` with the two managed identities that GitHub Actions signs in as, in North Central US.

1) The Azure region policy (above).
2) The Artifact Registry module did not wait for its API to be enabled. We added a `depends_on`.
3) The repository uses GitHub's immutable OIDC subject format (`repo:Owner@id/name@id:...`), so no CI binding matched. We added a `github_subject_prefix` variable to bootstrap.

IV. Deliverable 2: CI/CD Pipeline

The pipeline consists of four GitHub Actions workflows [5]. None of them stores a cloud credential. Every cloud action signs in through OIDC federation to the GCP or Azure identities created by bootstrap.

A. Continuous integration (`ci.yml`)

CI runs on every push to `main` and `development` and on every pull request. It has six jobs:

- **Python:** Ruff lint and format checks, and pytest against a PostgreSQL 16 service container (93 tests).
- **Docker images:** builds the images and runs a smoke test against the full docker-compose stack.
- **Terraform:** format, validate, and the mocked tests, with no credentials.
- **Kubernetes manifests:** kustomize and kubeconform, plus a namespace check.
- **Frontend:** ESLint, Vitest (30 tests), and a production build.
- **Frontend image:** builds the nginx image and checks the security headers it serves.

The latest run on `main` took 40 s and passed (Fig. 5).

B. Continuous delivery (`release.yml`)

Every push to `main` starts one Release run, shown in Fig. 6:

1) **ci** waits for CI to pass on the commit.
2) **review** prints a review sheet of every commit and migration since the last `prod-*` tag.
3) **plan** runs the real Terraform plan with a read-only identity.
4) **One approval** in the `prod` GitHub environment (three team reviewers; deploys only from `main`).
5) **release** applies exactly the reviewed plan. Then it backs up Cloud SQL if there are new migrations, builds and pushes the three images tagged with the commit SHA, runs migrations, rolls out ingest and search (undone automatically on failure), and deploys the frontend to Cloud Run with no traffic. Traffic moves to the new revision only after a health check passes. A final check calls the API through the load balancer.
6) **tag** creates `prod-YYYY-MM-DD.N` and a GitHub Release listing the commits.

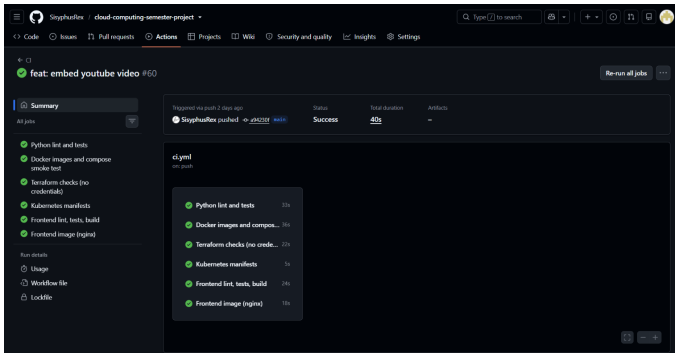


Fig. 5. A passing CI run on `main` with all six jobs.

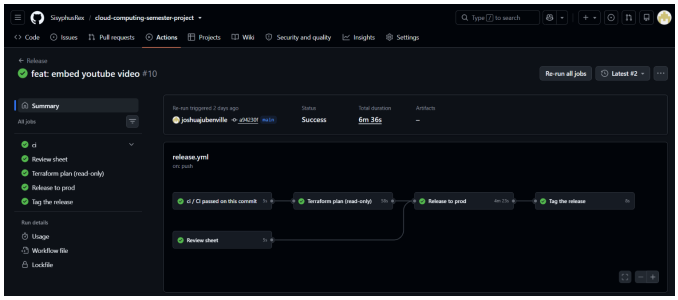


Fig. 6. A Release run: review, plan, the `prod` approval gate, release, and tag.

`rollback.yml` redeploys any earlier tag. `terraform.yml` posts a preview plan on pull requests. When the environment is paused, a release stops before asking for approval.

C. Evidence that it works

The first production release needed five runs. Table III shows that each failure stopped safely, before it changed anything it should not have. A failed frontend check left the previous revision serving traffic. After that, the pipeline released `prod-2026-10-19.1` (invite-only sign-in, the Admin page) and `prod-2026-10-27.1` (the embedded player). Both took one approval and about six minutes.

V. Deliverable 3: Basic Functionality

The prototype implements the workflow from the proposal, end to end:

1) A user signs in with Google (Fig. 7). The APIs verify the ID token's signature and audience, then look up the user's role. Only invited users can sign in.

TABLE III
FIRST RELEASE ATTEMPTS (2026-10-18)

Run	Failed at	Cause → fix
1	plan sign-in	Immutable OIDC subjects → subject prefix
2	migrations	ConfigMap had no namespace → `namespace: li` + check
3	pause check	Inferred pause from run 2's leftovers → explicit annotation
4	frontend check	Cloud Run reserves `/healthz` → `/health`
5	—	Released as `prod-2026-10-18.1`

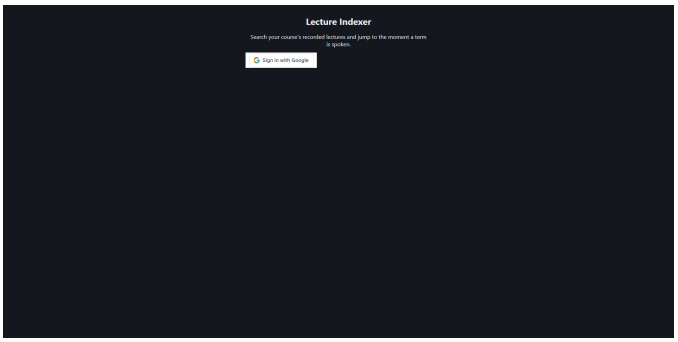


Fig. 7. Google sign-in on the production frontend.

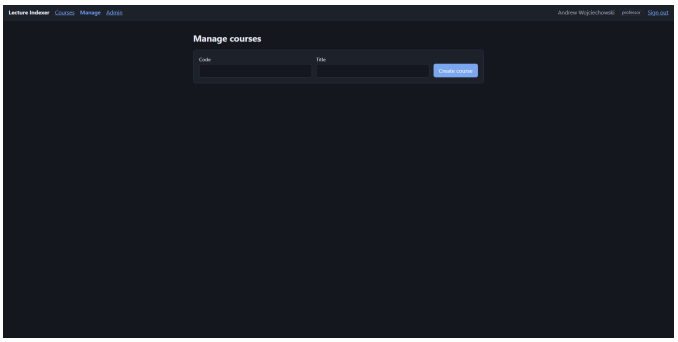


Fig. 8. Manage page, where professors create courses and register lectures.

2) A professor creates a course and registers a YouTube video (Fig. 8). The Ingest API checks that the video belongs to the professor's own channel and downloads the auto-generated caption track. It removes word-timing tags, rolling repeats, and sound tags such as `[Music]`, merges cues into segments of up to about 20 s, and stores them.
3) A student searches the course. The Search API ranks the matches with PostgreSQL full-text search and returns highlighted excerpts. Each result plays in an embedded YouTube player at its timestamp (Fig. 9).
4) An admin invites users and sets their roles (Fig. 10).

A. Manual testing with real Google accounts

Before deploying, we tested against the real Google and YouTube APIs with a professor account that owns a channel and a separate student account [6]. Table IV lists the cases.

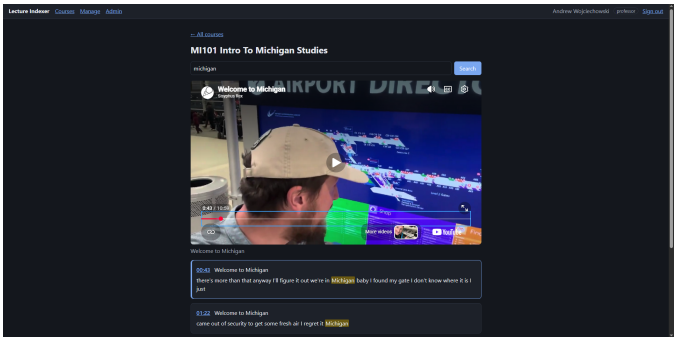


Fig. 9. Search results with highlighted excerpts and the embedded player at the matching timestamp.

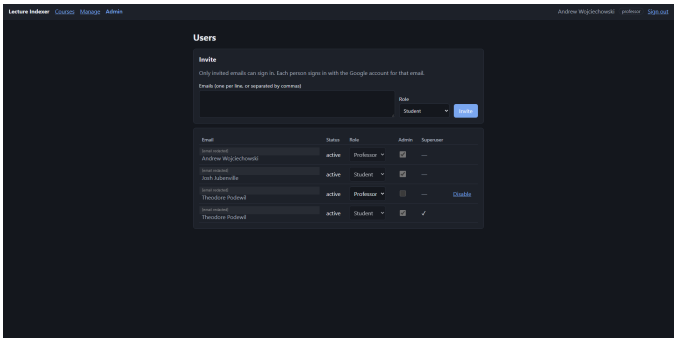


Fig. 10. Admin page: invitations, roles, and the admin and superuser flags (email addresses redacted).

TABLE IV
MANUAL TEST CASES (2026-10-18)

#	Case	Result
1	Google sign-in (professor)	Pass
2	Manage page loads	Pass
3	Register own video	Pass: 37 segments indexed
4	Search a spoken word	Pass: hit at 94 s
5	Re-register same video	Pass: idempotent, 37 segments
6	Register a video we do not own	Pass: 403, 2 quota units used
7	Uploaded captions only	Skipped; unit-tested
8	Delete a lecture	Pass: segments cascade-deleted
9	Student view	Pass: no Manage page, search works

On 2026-10-18 we repeated the main flow in production: sign in, register a video, search as a student, and follow the link. All of it passed.

B. Automated tests

The latest CI run passed 93 Python tests and 30 frontend tests. The Python tests cover caption parsing against a real VTT fixture, the YouTube client, routes, invitations, migrations, and query building. The frontend tests cover the API client, the pages, and the player. With the 3 Terraform tests, the suite has 126 tests.

C. Observations

Registering one lecture costs 252 YouTube API quota units, so the default quota of 10,000 units allows about 39 registrations a day. Searches use no quota. Because segments are up to 20 s long, a link opens on average about 10 s before the word is spoken. Shorter segments would make links more precise, at the cost of splitting phrases across segments.

VI. Security Already in Place

The full security review is due in Milestone 3. The prototype already has:

- Keyless CI and keyless workloads, through OIDC federation and Workload Identity.
- The database admin password in Secret Manager.
- IAM database users with least-privilege group roles: read-write for ingest; read-only for search, except claiming invitations on the `users` table.
- Cloud SQL reachable only on a private IP, and a default-deny NetworkPolicy.
- A Content Security Policy that allows connections only to the API and Google Sign-In.
- Invite-only sign-in with role-checked endpoints.

VII. Next Steps (Milestone 3)

For Milestone 3, we plan to add:

- A nightly backup CronJob that writes an encrypted `pg_dump` to immutable, cool-tier Azure Blob Storage.
- A Cloud Monitoring dashboard and alerts, built on the structured logs the services already emit (`http_request`, `youtube_api_call`).
- A written security review.
- Developer onboarding documentation.

References

[1] T. Podewil, J. Jubenville, and A. Wojciechowski, "Lecture Indexer: A multi-cloud service for timestamped keyword search of recorded lectures," CS 6030 Milestone 1 proposal, Western Michigan Univ., 2026.

[2] "ADR 0001: Host the frontend on Cloud Run instead of Azure Static Web Apps," https://github.com/SisyphusRex/cloud-computing-semester-project/blob/main/docs/architecture_decision_records/0001-frontend-on-cloud-run.md

[3] "ADR 0002: Invite-only sign-in, with roles set by admins in the database," https://github.com/SisyphusRex/cloud-computing-semester-project/blob/main/docs/architecture_decision_records/0002-invite-only-users-and-admin-roles.md

[4] "Bootstrap runbook" and "First deployment," https://github.com/SisyphusRex/cloud-computing-semester-project/tree/main/docs

[5] "Release process," https://github.com/SisyphusRex/cloud-computing-semester-project/blob/main/docs/release-process.md; releases: https://github.com/SisyphusRex/cloud-computing-semester-project/releases

[6] "Real Google/YouTube test: progress log," https://github.com/SisyphusRex/cloud-computing-semester-project/blob/main/docs/milestones/m2_prototype_review/real-youtube-test.md